

Fonctionnement du simulateur

ARM STM32

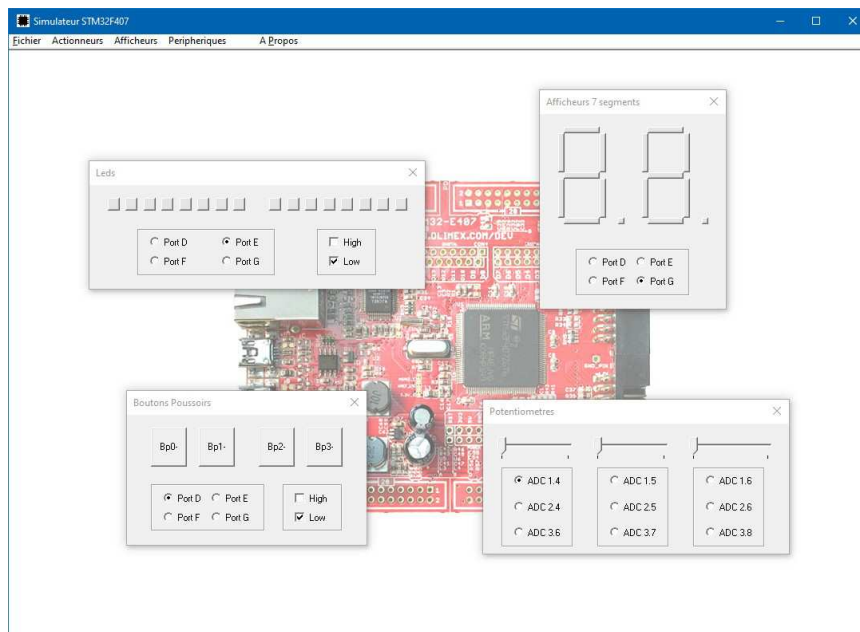
1 - Présentation du simulateur

Ce logiciel est un simulateur de microcontrôleur de type ARM STM32 destiné à un usage pédagogique. C'est un simulateur de haut niveau, qui travaille au niveau du programme en langage C, en s'appuyant sur des bibliothèques de simulation dont l'équivalent existe dans la réalité (Voir <https://github.com/joegil95/Libraries-for-Olimex-E407-STM32F407->), de sorte qu'il est très facile de transposer un programme réalisé dans le simulateur en un programme fonctionnant sur le véritable microcontrôleur.

Ce logiciel N'EST PAS un simulateur de bas niveau, et n'est donc pas fait pour simuler l'exécution d'un code compilé. De tels simulateurs existent à ce jour mais sont très incomplets vu la complexité et la rapidité des ARM 32 bits, ce qui rend la chose difficile.

Le microcontrôleur simulé est précisément un STM32F407, fabriqué par la société franco-italienne ST-Microelectronics, qui fait partie de la famille STM32F4, et équipe les cartes Olimex E-407 et Discovery STM32F407.

La mémoire programme n'est pas limitée dans le simulateur contrairement au vrai microcontrôleur.



Exemple d'interface utilisant des boutons, des leds, un afficheur à segments et un potentiomètre

Pour utiliser le simulateur, il faut disposer d'une version de Visual Studio 2015 community, permettant la programmation en C pour l'utilisateur, et en C++ pour le simulateur lui-même. Ce logiciel de développement est téléchargeable en ligne et nécessite seulement un enregistrement sur un compte Microsoft.

Il est alors possible d'écrire un programme en C (qui doit néanmoins pour des raisons techniques porter une extension cpp), de tester ce programme au sein d'une interface graphique en utilisant des actionneurs, afficheurs et périphériques virtuels. Il est même possible de faire du débogage, du pas à pas et de positionner des points d'arrêt dans le programme.

Il faut toutefois être conscient que l'interface graphique Windows est infiniment plus lente que le matériel réel, ce qui induit des limitations. Les timings dans les temporisations ou interruptions doivent par exemple être adaptés. D'une manière générale, un programme fonctionnant dans le simulateur devrait fonctionner dans la réalité. L'inverse peut nécessiter quelques aménagements pour les raisons qui précèdent.

Voir plus loin pour la création / configuration d'un projet VS-2015.

2- Présentation du microcontrôleur simulé

Le microcontrôleur STM32F407ZG est un microcontrôleur 32 bits doté d'un cœur ARM Cortex-M4 avec calculs en virgule flottante. L'architecture ARM (Advanced Risc Machine) est un concept développé par ARM Ltd depuis les années 1990, qui est vendu sous forme de licences à de nombreux fondeurs de composants, ici ST-Microelectronics.

Ce microcontrôleur dispose de :

1 Mo de mémoire programme (Flash)
192 Ko de mémoire de données (SRAM)
4 Ko de mémoire de sauvegarde (Backup SRAM)

7 ports GPIO nommés A à G de 16 bits chacun, dont seuls les ports D à G sont réellement utilisables, les autres étant affectés à divers périphériques.

Fréquence de fonctionnement jusqu'à 168 Mhz avec un quartz externe à 12 MHz et des PLL intégrées dans le composant. Alimentation en 3,3 volts typiquement.

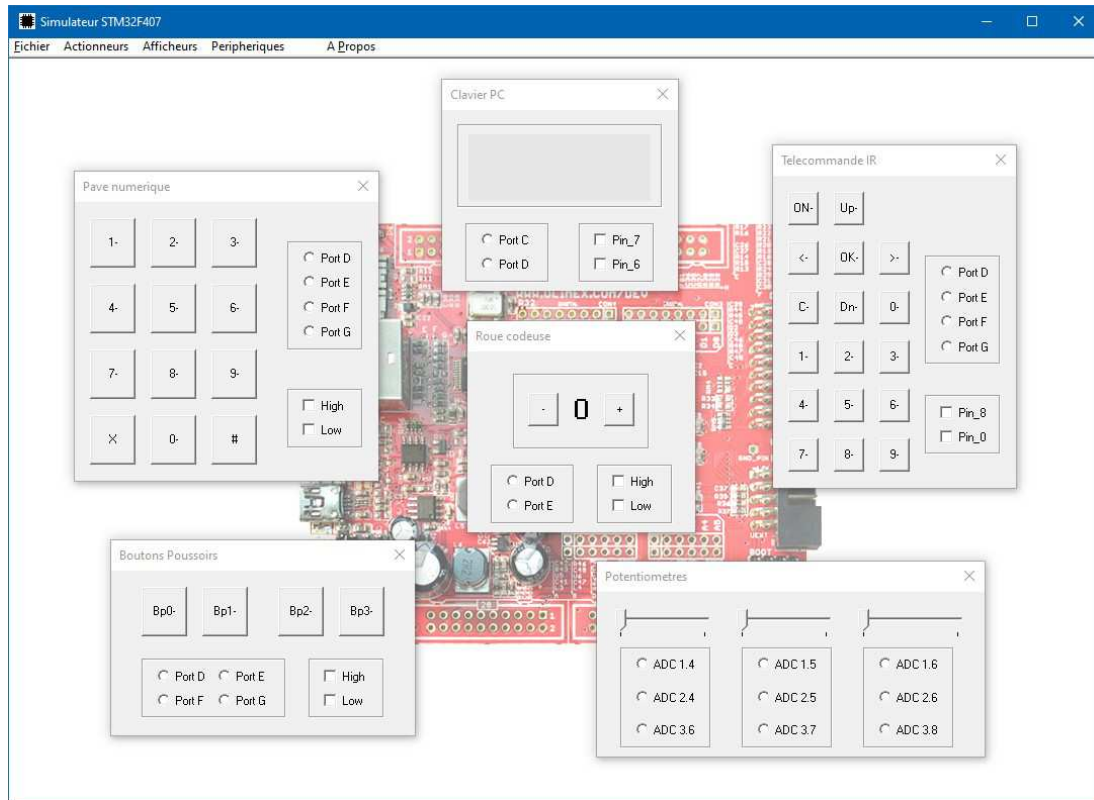
Périphériques du microcontrôleur utilisables dans le simulateur

- ADC 12 bits à 16 canaux
- DAC 12 bits
- Interfaces série SPI
- Interfaces de communication I²C
- Interface de communication USART
- 4 timers 16 ou 32 bits avec interruptions
- Backup SRAM de 4 KO
- Horloge RTC intégrée
- Interface SDIO pour cartes SD
- Port Ethernet 10 / 100 Mb
- Générateur de nombres aléatoires

Accessoires et périphériques virtuels disponibles dans le simulateur

- 4 ports 16 bits D, E, F, G
- 4 boutons poussoirs
- 16 leds (8 vertes et 8 rouges)
- 1 buzzer
- 3 potentiomètres
- 1 pavé numérique 12 touches

- 1 roue codeuse à 10 positions
- 1 clavier PC
- 1 télécommande IR
- 1 afficheur 2 x 7 segments à anodes communes
- 1 afficheur LCD 4 chiffres fonctionnant sur SPI
- 1 afficheur texte 16 x 2 classique
- 1 afficheur graphique 128 x 64 pixels



Actionneurs disponibles dans l'interface : boutons, potentiomètres, pavé numérique, roue codeuse, télécommande, clavier PC

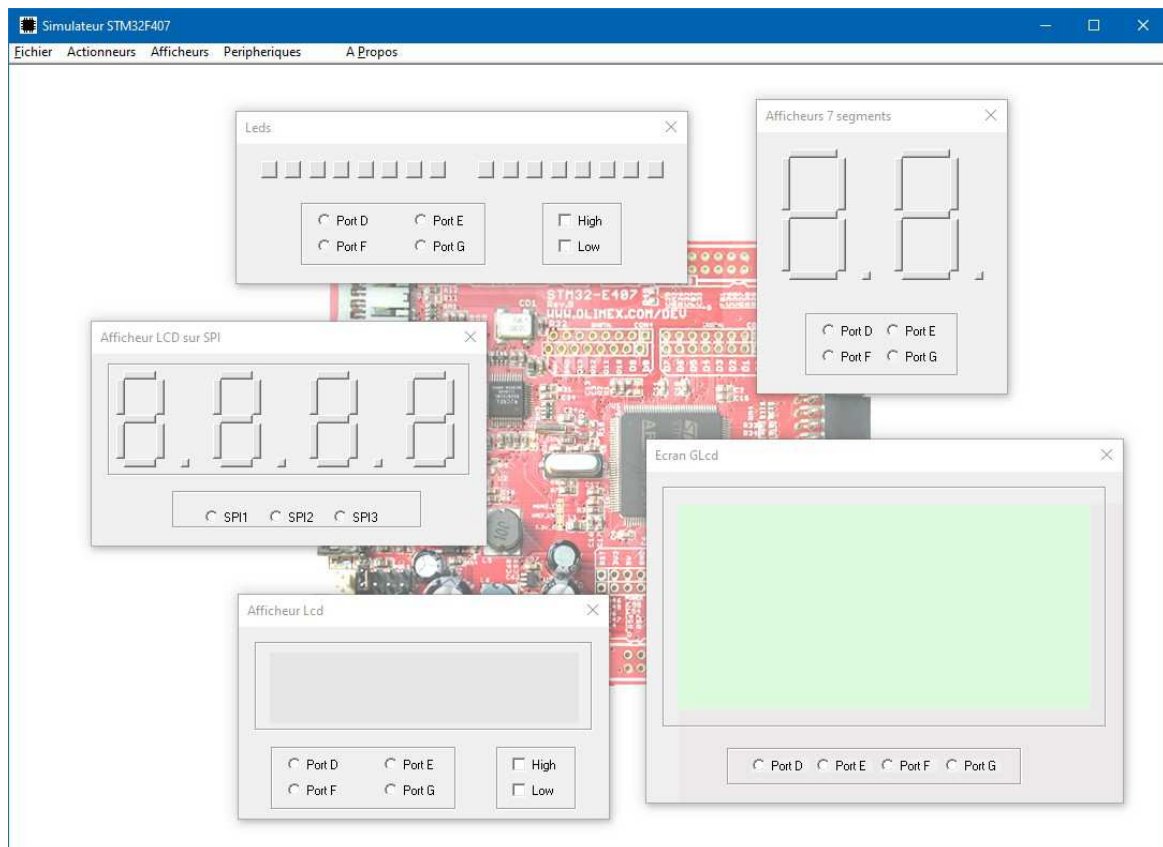
- 1 horloge RTC interne calée sur celle de l'ordinateur
- 1 horloge RTC externe de type DS1307 fonctionnant en I²C
- 1 EEPROM externe AMD fonctionnant sur SPI
- 1 lecteur de carte SD externe sur port SDIO
- 1 port réseau Ethernet connectable à celui de l'ordinateur
- 1 port série connectable au port série de l'ordinateur

Il est possible et même souhaitable d'enregistrer la configuration des périphériques, une fois celle-ci bien établie, afin de pouvoir la restaurer rapidement à chaque lancement.

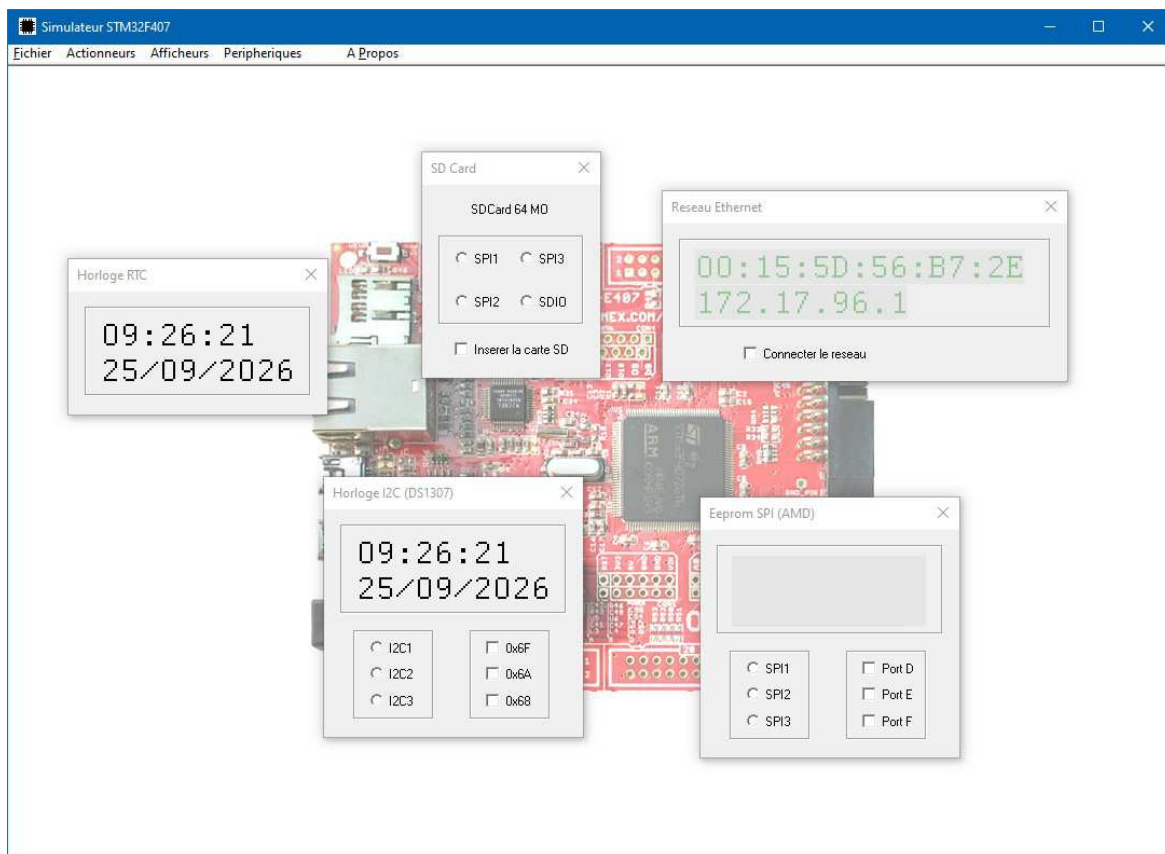
La sauvegarde de la configuration se fait dans le fichier SimulArm.ini qui se trouve dans le dossier C:\Etc

Si ce dossier n'existe pas, il faudra le créer pour utiliser les possibilités de sauvegarde.

En plus des types de base des langages C / C++, le simulateur utilise des types de variables fortement typés : int8_t, int16_t, int32_t (non signés) et uint8_t, uint16_t, uint32_t (signés) qui sont très répandus par ailleurs.



Afficheurs disponibles dans l'interface : leds, afficheur à segments, afficheur 4 chiffres sur SPI, Afficheur LCD 16x2, afficheur graphique 64x128



Périphériques disponibles dans l'interface : Horloge RTC interne, Horloge RTC externe DS1307, Eeprom externe, accès au réseau Ethernet, lecteur de carte SD

3 – Configuration et gestion des ports d’entrées - sorties

Le simulateur comporte 4 ports GPIO 16 bits nommés D, E, F, G, qui sont accessibles via la librairie Lib_Gpio.cpp, associée à Lib_Gpio.h

La configuration d’un port GPIO se fait à l’aide d’une fonction nommée LibGPIO_Conf() qui nécessite plusieurs paramètres qui sont :

Champ	Signification	Valeurs possibles
<i>GPIOx</i>	Port	<i>GPIOD ... GPIOG</i>
<i>GPIO_Pins</i>	Broches	<i>GPIO_Pin_0 ... GPIO_Pin_15</i> <i>GPIO_Pin_Low ... Hig ... All</i>
<i>GPIO_Mode</i>	Entrées / Sorties / Alternatives	<i>GPIO_Mode_OUT - GPIO_Mode_IN</i> <i>GPIO_Mode_AF</i>
<i>GPIO_Type</i>	Sorties Push-pull / Open-drain Entrées avec ou sans Pull up / down	<i>GPIO_OType_PP - GPIO_OType_OD</i> <i>GPIO_PuPd_NOPULL -</i> <i>GPIO_PuPd_UP - GPIO_PuPd_DOWN</i>
<i>GPIO_Speed</i>	Horloge	<i>GPIO_Speed_2MHz ... GPIO_Speed_50MHz</i>

Exemples:

Pour configurer des entrées associées à des boutons poussoirs:

```
LibGPIO_Conf(GPIOE, GPIO_Pin_0 | GPIO_Pin_1, GPIO_Mode_IN, GPIO_PuPd_UP,  
GPIO_Speed_2MHz);
```

Pour configurer des sorties associées à des leds:

```
LibGPIO_Conf(GPIOD, GPIO_Pin_Low, GPIO_Mode_OUT, GPIO_OType_PP,  
GPIO_Speed_2MHz);
```

La librairie Lib_Gpio.cpp offre également des fonctions de manipulation des entrées - sorties :

GPIO_ReadInputData(), GPIO_SetBits(), GPIO_ResetBits(), GPIO_Write()

Exemple:

```
x= GPIO_ReadInputData(GPIOE);  
GPIO_Write(GPIOD, 0x00FF);  
GPIO_SetBits(GPIOD, GPIO_Pin_7 | GPIO_Pin_8);
```

On notera que les boutons poussoirs fonctionnent en logique inverse et nécessitent l’activation des résistances de Pull-up. Ils sont reliés aux 4 bits de poids faible du demi-port sélectionné.

Les leds fonctionnent en logique normale. Elles ne sont connectables que par lot de 8 leds sur un demi-port.

Pour utiliser des boutons poussoirs ou des leds, il faut lancer le simulateur, afficher et configurer la boîte de dialogue “Boutons” ou “Led”, puis lancer la simulation.

Voir démo TEST_Gpio.cpp

4 - Configuration des afficheurs

a – Configuration et utilisation de l’afficheur 2 x 7 segments

L’afficheur 2 x 7 segments est connectable aux ports D à G. On utilisera de préférence le port G. Il utilise les 8 bits de poids faible pour les 7 segments et le point, plus (facultativement) le premier bit de poids fort comme sélecteur entre le côté gauche et le côté droit. Il n’est en effet pas possible d’utiliser les deux chiffres en même temps, mais un astucieux multiplexage temporel avec ce sélecteur permet de simuler un affichage sur 2 chiffres simultanés. L’idéal étant pour cela d’utiliser un timer et des interruptions.

Un tableau de codes doit être utilisé pour obtenir des chiffres sur l’afficheur, mais il n’y a pas de librairie spécifique pour cet afficheur.

Pour utiliser cet afficheur, il faut lancer le simulateur, afficher et configurer la boîte de dialogue “Afficheurs 7 segments”, puis lancer la simulation.

Voir démo TEST-7-Segt.cpp

b – Configuration et utilisation de l’afficheur LCD 16x2

L’afficheur LCD 16x2 (2 lignes de 16 caractères) est connectable aux ports D à G.

Il utilise les 8 bits de poids faible ou fort du port selon la sélection, dont 4 bits pour les données.

Les principales fonctions disponibles dans la librairie Lib_Lcd sont *LCD_Init()* pour la configuration, qui s’occupe de tout, *LCD_Erase()* pour effacer, *LCD_MoveCursor()* pour se déplacer, *LCD_Write()* pour afficher du texte et *LCD_ShowInt()* pour afficher des valeurs.

Pour utiliser cet afficheur, il faut lancer le simulateur, afficher et configurer la boîte de dialogue “Afficheur LCD 16x2”, puis lancer la simulation.

Voir démo TEST-Lcd-16x2.cpp

c – Configuration et utilisation de l’afficheur LCD graphique 64x128

L’afficheur LCD graphique 64x128 pixels est connectable aux ports D à G.

Il utilise les 16 bits du port sélectionné.

Une fonte de caractères 5x8 pixels est fournie pour afficher texte, chiffres et autres symboles.

Les principales fonctions disponibles dans la librairie Lib_Glcd sont *GLCD_InitDisplay()* pour la configuration, qui s’occupe de tout, *GLCD_ClearScreen()* pour effacer, *GLCD_Move()* pour se déplacer, *GLCD_Write()* pour afficher du texte et *GLCD_ShowInt()* pour afficher des valeurs.

L’afficheur est en fait composé de deux afficheurs 64x64 juxtaposés. Les pixels ne sont adressables que par paquet de 8 organisés verticalement. Un tel paquet de pixels est référencé par 3 informations utilisés par *GLCD_Move()* : le côté gauche= LEFT ou droite= RIGHT, la page ou ligne= 0 à 7, et enfin la colonne= 0 à 63. Une page ou ligne fait 8 pixels de haut.

L’effacement complet de l’afficheur est lent, dans le simulateur comme dans la réalité, et doit être utilisé avec modération.

Pour utiliser cet afficheur, il faut lancer le simulateur, afficher et configurer la boîte de dialogue “Afficheur GLCD 64x128”, puis lancer la simulation.

Voir démo TEST-Glcd.cpp

5 - Configuration des accessoires et périphériques principaux

a – Configuration et utilisation des ADC

Le simulateur intègre 3 Convertisseurs Analogique – Numérique nommés ADC1, ADC2 et ADC3 à 16 canaux chacun (0 à 15) en modes 8, 10 ou 12 bits connectables à des potentiomètres.

Les fonctions disponibles dans la librairie Lib_Adc sont *LibADC_Init()* pour la configuration, qui s'occupe de tout, et *LibADC_Read()* pour l'acquisition.

Exemple :

```
LibADC_Init(ADC1, ADC_Channel_5, ADC_Resolution_8b);  
x= LibADC_Read(ADC1, ADC_Channel_5);
```

Pour utiliser un ADC, il faut lancer le simulateur, afficher et configurer la boîte de dialogue “Potentiomètres”, puis lancer la simulation.

Voir démo TEST-Adc.cpp

b – Configuration et utilisation du SPI et de l'afficheur LCD à 4 chiffres

Le simulateur ne propose pas d'interface SPI générale, mais offre un afficheur LCD fonctionnant sur SPI1, SPI2 ou SPI3, qui permet de se familiariser avec ces périphériques de communication synchrone utilisables en mode maître. Ils fonctionnent en modes 8 ou 16 bits à une cadence réglable à l'aide d'une prédivison. Cette cadence est très théorique dans le simulateur.

Les fonctions disponibles dans la librairie Lib_Spi sont *LibSPI_Init()* pour la configuration, qui s'occupe de tout, et *LibSPI_Send()* pour l'émission sur le SPI.

Exemple :

```
LibSPI_Init(SPI2, LibSPI_DataSize_8b, SPI_BaudRatePrescaler_256);  
LibSPI_Send(SPI2, 0x45);
```

Un tableau de codes doit être utilisé pour obtenir des chiffres.

Pour fonctionner correctement dans la réalité et ne pas figer les cristaux, ces afficheurs à cristaux liquides doivent recevoir en alternance les codes et leurs compléments. La chose n'est pas indispensable mais est supportée par l'afficheur virtuel.

Pour utiliser un afficheur sur SPI, il faut lancer le simulateur, afficher et configurer la boîte de dialogue “Afficheur LCD sur SPI”, puis lancer la simulation.

Voir démo TEST-Spi+Lcd.cpp

c – Configuration et utilisation de l'I2C

Le simulateur ne propose pas d'interface I2C générale, mais il comprend une horloge RTC de type DS1307 connectable sur I2C1 avec l'adresse 0x68, qui permet de se familiariser avec ce périphériques de communication synchrone utilisable en mode maître. La fréquence de fonctionnement est très théorique dans le simulateur.

Les fonctions disponibles dans la librairie Lib_I2C sont *LibI2C_MasterInit()*, *LibI2C_GetReg()*, *LibI2C_GetMultiReg()*, *LibI2C_SetReg()* et *LibI2C_SetMultiReg()*

Exemple :

```
LibI2C_MasterInit(I2C1, 100000);           // 100 kHz
LibI2C_SetReg(I2C1, adresse, registre, valeur);
valeur= LibI2C_GetReg(I2C1, adresse, registre);
```

Pour utiliser l'horloge DS1307 sur I2C, il faut lancer le simulateur, afficher et configurer la boîte de dialogue "Horloge I2C (DS1307)", puis lancer la simulation.

Voir démo TEST-I2c.cpp

d – Configuration et utilisation des Timers

Dans son utilisation la plus simple, un timer est un compteur indépendant, qui s'incrémente à une fréquence paramétrable et peut être associé à une interruption pour déclencher des actions périodiques.

Le simulateur intègre 4 timers simples nommés TIM2 à TIM5.

Les timers TIM3 et TIM4 fonctionnent sur 16 bits, tandis que TIM2 et TIM5 sont sur 32 bits. Cela influe sur la valeur maximale jusqu'à laquelle ils peuvent compter. Au-delà ils repassent à 0, mais en général on leur impose une valeur maximale inférieure qui détermine leur période.

La librairie Lib_Timer offre les fonctions *LibTimer_Init()*, pour configurer un timer, et *LibTimer_Interrupts()*, pour activer l'interruption associée si nécessaire.

Exemple :

```
LibTimer_Init(TIM2, 1000, 100);           //   Prédivision par 1000, Période 100 cycles
LibTimer_Interrupts(TIM2, ENABLE);
```

La fréquence de comptage des timers est très théorique dans le simulateur. Ils s'appuient sur les timers internes de Windows, dont la précision est seulement d'une milliseconde. Donc toute fréquence donnant une période inférieure à 1 ms sera ramenée à 1 ms.

Voir démo TEST_Timer+Interrupts.cpp

e – Gestion des Interruptions associées aux timers

Les seules interruptions gérées dans le simulateur sont celles associées au déclenchement d'un timer. Ces interruptions sont associées à des routines d'interruptions. A chaque fois que l'interruption se déclenche, la routine est exécutée de façon prioritaire, interrompant brièvement le cours normal du programme. La routine doit acquitter l'interruption afin de rendre possible la suivante.

Exemple de routine d'interruption pour le Timer 2:

```
void TIM2_Handler()
{
    if (TIM_GetITStatus(TIM2, TIM_IT_Update) != RESET)    // vérification
```

```

    {
        // mettre ici le code à exécuter

        TIM_ClearITPendingBit(TIM2, TIM_IT_Update); // acquittement
    }
}

```

Il est toujours possible de vérifier le fonctionnement d'une routine d'interruption en y plaçant un breakpoint lors de l'exécution.

Voir démo TEST-Timer+Interrupts.cpp

f – Configuration et utilisation des DAC

Le simulateur intègre 2 Convertisseurs Numérique - Analogique nommés DAC1 et DAC2 fonctionnant sur 12 bits, connectables à l'afficheur LCD à 4 chiffres. Un DAC convertit une valeur numérique en tension proportionnelle, qui est visible sur l'afficheur. Le DAC1 utilise SPI1 et le DAC2 utilise SPI2.

Les fonctions disponibles dans la librairie Lib_Dac sont *LibDAC_Init()* pour la configuration, qui s'occupe de tout, et *LibDAC_Output()* pour fixer la valeur de sortie.

Exemple :

```

LibDAC_Init(DAC1);
x= LibDAC_Output(DAC1, valeur);

```

Pour utiliser un DAC, il faut lancer le simulateur, afficher et configurer la boîte de dialogue "Afficheur LCD sur SPI", puis lancer la simulation.

Voir démo TEST-Dac.cpp

g – Configuration et utilisation de l'UART

Le simulateur possède une UART utilisable pour la communication série asynchrone, connectable à un port série physique de l'ordinateur. On peut utiliser UART1 à UART6, selon le numéro du port COM de l'ordinateur. Ce port COM peut-être un port virtuel associé à un convertisseur USB/Série.

La vitesse, le nombre de bits, la parité et le nombre de bits de stop sont paramétrables.

Les données envoyées sont transmises sur le port série. Les données reçues peuvent être visualisées sur un afficheur.

Les fonctions disponibles dans la librairie Lib_Uart, qui ne prend pas en charge les contrôles de flux, sont *LibUart_Init()*, *LibUart_Getc()* et *LibUart_Putc()*

Exemple :

```

LibUart_Init(USART1, 19200, USART_WordLength_8b, USART_Parity_No,
            USART_StopBits_1);
LibUart_Putc(USART1, 'a');
c= LibUart_Getc(USART1);

```

Voir démo TEST-Uart.cpp

h - Configuration et utilisation du pavé numérique

Le simulateur comprend un pavé numérique ou keypad à 12 touches comportant les chiffres de 0 à 9, x et #.

Il est connectable aux parts D à G et utilise un demi-port. Les colonnes des touches sont reliées à des sorties et les lignes à des entrées. L'appui sur une touche met en relation la ligne et la colonne de la touche, de sorte qu'avec un petit algorithme il est possible de déterminer la touche sur laquelle on a appuyé.

Les fonctions disponibles dans la librairie Lib_Keypad sont *LibKeypad_Init()* pour l'initialisation et *LibKeypad_Get()* qui retourne la touche pressée. Les chiffres renvoient leur valeur, x renvoie 10 et # renvoie 11. Si aucune touche n'est pressée, la fonction retourne -1, ce qui impose d'utiliser une variable signée.

Pour utiliser le pavé numérique, il faut lancer le simulateur, afficher et configurer la boîte de dialogue "Pavé numérique", puis lancer la simulation.

Voir démo TEST-Keypad.cpp

i - Configuration et utilisation de la roue codeuse

Le simulateur comprend une roue codeuse, appareil rotatif qui permet de sélectionner une valeur de 0 à 9 en fonction de sa position.

Elle est connectable aux parts D et E et utilise seulement les 4 broches de poids faible, qui doivent être configurées en entrées avec pull-ups.

Dans la réalité, la roue codeuse passe par des états instables lors du changement de position, de sorte qu'il est nécessaire de prévoir une temporisation anti-rebonds.

Il n'y a pas de librairie spécifique pour la roue codeuse.

Pour utiliser la roue codeuse, il faut lancer le simulateur, afficher et configurer la boîte de dialogue "Roue codeuse", puis lancer la simulation.

Voir démo TEST-Roue.cpp

j - Configuration et utilisation de la télécommande infra-rouge

Le simulateur comprend une télécommande IR avec des chiffres et des flèches.

Elle utilise une broche 0 ou 8 d'un port D à G.

La communication se fait en série, bit par bit, selon un chronogramme spécifique au protocole NEC. Elle fonctionne, mais les timings sont très dépendants de la vitesse du PC sur lequel tourne le simulateur...

Voir protocole NEC pour en savoir plus.

Pour utiliser la télécommande IR, il faut lancer le simulateur, afficher et configurer la boîte de dialogue "Télécommande IR", puis lancer la simulation.

Voir démo TEST-Remote.cpp et librairie Lib_Remote

6 - Configuration des périphériques sophistiqués

a - Configuration et utilisation de l'horloge RTC interne

Le simulateur propose, comme le microcontrôleur réel, une horloge interne. La principale différence est que l'horloge virtuelle ne nécessite pas de mise à l'heure, étant automatiquement calée sur l'horloge de l'ordinateur. Elle offre la date, l'heure et un mécanisme d'alarme, en utilisant des structures pour la manipulation des informations.

On notera que les informations sont codées dans l'horloge au format BCD sur 8 bits et doivent donc être décodées.

Les principales fonctions disponibles dans la librairie Lib_Clock sont *LibClockt_Init()* qui doit être appelée, même si les date et heure fournies sont sans utilité, *LibClock_GetTime()* et *LibClock_GetDate()* pour récupérer les informations via des structures.

L'alarme peut également être utilisée.

Voir librairie LibClock qui comprend la définition des structures à utiliser pour communiquer avec l'horloge interne.

Pour utiliser l'horloge interne, il faut lancer le simulateur, afficher et configurer la boîte de dialogue "Horloge RTC interne", puis lancer la simulation.

Voir TEST-Clock-RTC.cpp

b - Configuration et utilisation du clavier PC

Windows dispose d'un clavier virtuel nommé OSK (On Screen Keyboard) manipulable à la souris et qui peut se substituer au clavier physique.

Il est possible de connecter ce clavier au simulateur en modélisant un ancien clavier de type PS/2, qui utilisait une liaison de type USRT. La ligne d'horloge PS/2 est connectée à la broche 9 du port G, tandis que la ligne de données est configurable sur la broche 6 ou 7 du port C ou D.

Pour utiliser le clavier PS/2, il faut lancer le simulateur, afficher et configurer la boîte de dialogue "Clavier PC", puis lancer la simulation. Normalement le clavier virtuel Windows doit apparaître (Sinon, le lancer manuellement). La boîte de dialogue affiche les scan codes générés par l'appui sur les touches du clavier, qu'il faut alors traduire en caractères.

Voir librairie Lib_Ps2 pour en savoir plus. L'utilisateur a seulement à utiliser *LibPs2_Init()* pour l'initialisation et *LibPs2_Get()* pour récupérer les scan-codes. Cette dernière renvoie -1 si aucune touche n'a été pressée.

Voir TEST-Clavier-Ps2.cpp

c - Configuration et utilisation du port réseau

Le simulateur permet de se connecter au port Ethernet de l'ordinateur, d'émettre et de recevoir des trames. La boîte de dialogue réseau du simulateur affiche l'adresse IP v4 et l'adresse MAC de l'ordinateur, avec une case à cocher pour se connecter.

L'accès au réseau passe par la librairie LibPcap, et il est donc nécessaire d'installer ce logiciel libre sur l'ordinateur avant de lancer le simulateur.

Les fonctions disponibles dans la librairie Lib_Ethernet sont *LibEthernet_Init()* qui vérifie que la connexion à LibPcap a réussi, *LibEthernet_Send()* qui permet d'émettre des trames sur le réseau, *LibEthernet_Recv()* qui permet d'en recevoir, et *LibEthernet_Close()* pour terminer.

Il s'agit de trames de bas niveau, dont la manipulation s'adresse à des programmeurs avertis...

Voir démo TEST-Ethernet.cpp

d - Configuration et utilisation du lecteur SD-Card

Le simulateur dispose d'un lecteur de carte SD virtuel simulant une carte SD de 64 MO utilisée en mode SDIO sur 4 bits. Pour fonctionner, le simulateur doit pouvoir accéder à un « ramdisk » ou lecteur virtuel (S : par exemple) de 64 MO en mode FAT créé avec un logiciel externe dédié comme SoftPerfect_Ramdisk.

Lors du clic sur la case à cocher « Insérer la carte SD », le simulateur tente d'exécuter ce programme via Rmdiskc.bat qui doit se trouver dans le même dossier. A défaut, créer ce lecteur manuellement en parallèle.

Une fois l'accès à la carte SD initialisé avec *SD_Init()*, il est possible de monter un filesystem de type FATFS (à déclarer) sur cette carte avec *f_mount()* et de manipuler des fichiers avec les autres fonctions de la librairie Lib_Sdcard.

Les fichiers ainsi créés sont aussi accessibles dans le lecteur correspondant (S :) de l'ordinateur.

Voir démo TEST-SdCard.cpp

7 – Noyau multitâche

Le simulateur offre un noyau multitâche coopératif (Kernel) qui permet de faire de la programmation multitâche, dans laquelle plusieurs tâches s'exécutent en parallèle, avec une rotation des tâches. Cette façon de faire permet de gérer plus facilement des applications sophistiquées ayant beaucoup de choses à faire « en même temps ».

Ce noyau multitâche est assez rudimentaire mais efficace. Il peut gérer jusqu'à 10 tâches parallèles ayant chacune un code séparé. Pour utiliser ce noyau multitâche, il est nécessaire d'inclure dans le projet le fichier Lib_Kernel.obj et de s'appuyer sur TEST-Kernel.cpp qui illustre le fonctionnement de 3 tâches parallèles.

Le noyau multitâche permet de moduler la priorité des tâches dans la rotation, de mettre une tâche en pause durant un certain temps, et d'échanger des informations entre tâches.

8 – Configuration des projets sous Visual Studio 2015

Le simulateur a été testé sous Visual Studio 2010 et 2015.

Dans les deux cas, il est nécessaire de créer un projet avec des options spécifiques.

Créer un projet vide de type Win32 avec support des MFC (pour l'interface graphique)

- Insérer dans le projet toutes les librairies du dossier Lib et le fichier res
- Insérer dans le projet les librairies nécessaires du dossier Src
- Ajouter le programme principal, par exemple un des programmes de Test, à placer dans Src

Modifier ensuite les propriétés suivantes du projet :

- Utiliser les MFC dans une DLL partagée, dans « Général »
- Désactiver le jeu de caractères Unicode, dans « Général »
- Désactiver les Entêtes précompilés, dans « C/C++ > Headers précompilés »
- Désactiver toutes les optimisations, dans « C/C++ > Optimisations »
- Désactiver les fonctions intrinsèques, dans « C/C++ > Optimisations »

Lors de l'utilisation du noyau multitâche, modifier en plus les propriétés suivantes:

- Vérifications de base à l'exécution = Défaut, dans « C/C++ > Génération »